SEION MATH CORE V3: A REPRODUCIBLE VERIFICATION SUITE FOR TYPED MULTILINEAR TREE CERTIFICATES

ELIUTH CHAVERO JASSO

Abstract. This companion specifies the software and artifact protocol used to evaluate finite typed multilinear composition trees under recursive orthogonal projection. The implementation separates explicit-coordinate and vectorized NumPy evaluators, a float64 CUDA evaluator, exact ordered-tree enumeration, dynamic-programming certificates, directed interval lower constructions, independent gradient and derivative-free search, signed forests, and CP/projection budgets. The base registry contains 15,493 deduplicated scientific instances, 81,445 enumerated tree occurrences, and 1,530 exhaustive leakage-mask evaluations; the maximum registered CPU/GPU difference is 1.922×10^{-8}. Every run records source, resolved configuration, mathematical-object and input hashes, tensors, hardware, environment, metrics, certificates, logs, and artifact hashes. The companion documents a resource-gated extended optimizer grid rather than presenting unexecuted seeds as evidence. It makes no mathematical novelty claim and does not substitute software completeness for independent theorem review.

1. Scope and separation from the mathematics paper

The mathematical manuscript proves finite error identities and upper bounds. This companion answers a different question: can the declared finite objects, certificates, experiments, figures, tables, and PDFs be regenerated from one repository with failures retained? It describes interfaces and evidence contracts, not theorem novelty. The strict status remains `FAIL_CLOSED_NOVELTY` because novelty, human approval, and the extended optimizer grid are unresolved.

Date: 29 July 2026.

2020 *Mathematics Subject Classification.* 68N30, 65Y05, 65G50, 15A69.

Key words and phrases. research software, multilinear tensor, typed tree, reproducibility, CUDA parity, interval arithmetic.

Software/reproducibility companion. Contact metadata and release authorization remain pending author and human-review gates.

Table 1. Generated enumeration summary.

grammar	occurrences	unique hashes	internal nodes	max depth
binary	2055	2055	1–8	8
mixed_2_3_4	78879	78879	1–5	5
quaternary	167	167	1–4	4
ternary	344	344	1–5	5

2. Repository architecture

path	role
`src/seion_core/research_v3/types.py`	finite typed Hilbert spaces, isometries, and orthogonal projectors
`src/seion_core/research_v3/typed_tree.py`	immutable leaves/nodes, validation, canonical serialization, and hashes
`src/seion_core/research_v3/tree_enumeration.py`	exact ordered binary, ternary, quaternary, and mixed-arity enumeration
`src/seion_core/research_v3/exact_evaluation.py`	explicit-coordinate and NumPy ambient evaluators
`src/seion_core/research_v3/projected_evaluation.py`	recursive projection, named root errors, and Torch CPU/CUDA evaluator
`src/seion_core/research_v3/certificates.py`	homogeneous, nodewise, mixed-mask, and path-sum certificates
`src/seion_core/research_v3/telescoping_order.py`	pair-exchange cost and deterministic optimal ordering
`src/seion_core/research_v3/interval_certification.py`	directed interval recursion for explicit lower constructions
`src/seion_core/research_v3/adversarial_search.py`	Adam/L-BFGS and derivative-free empirical lower search
`src/seion_core/research_v3/run_schema.py`	immutable-style run identity and mandatory artifact contract

All v3 additions remain under this repository. Historical v1/v2 artifacts are not overwritten.

3. Typed tree API

A `TypedSpace` validates the ambient dimension, reduced dimension, isometry, and projector identities. A `TypedLaw` carries an ordered input signature, output type, and dense tensor. A `Leaf` carries a label and type; a `Node` carries a law identifier, output type, and ordered children. Validation occurs before evaluation:

(1) every leaf label resolves to reduced input data of the declared size;
(2) every child output type equals the corresponding law input type;
(3) every law tensor shape agrees with its signature; and
(4) every isometry and projector passes dimension and orthogonality checks.

Invalid edges are negative controls, not failed floating-point contractions.

4. Exhaustive enumerator

The enumerator uses exact recursive grammars with canonical ordered and unordered shape identifiers. It records depth, leaf-depth statistics, imbalance, Strahler number, path-length sum, automorphism count, repeated subtrees, and expression-DAG compression. Hash uniqueness is checked within each grammar; cross-grammar reuse is intentional when a mixed grammar contains a pure binary, ternary, or quaternary tree.

Table 2. Generated CPU/GPU parity summary.

precision	cases	max abs. difference	median difference	result
complex128	4	0	0	pass
complex64	4	1.92×10^{-8}	5.60×10^{-9}	pass
float32	4	1.92×10^{-8}	5.60×10^{-9}	pass
float64	4	0	0	pass

5. Evaluation engines and parity

The coordinate reference evaluator applies each tensor with explicit index loops. The independent NumPy path contracts modes through Einstein/tensordot operations. Recursive projection is applied after every node law. The Torch path disables TF32 and mixed precision for the float64/complex128 parity contract.

Tests cover real and complex data, arities two through four, repeated leaves, invalid signatures, exact root identities, and CPU/GPU parity. The benchmark uses an exactly invariant control so reference, NumPy, and CUDA semantics coincide.

6. Certificate engine

At each node the engine carries full/projected magnitudes and ambient/projected/normal errors. It can use:

(1) the homogeneous k and $k-1$ certificates;
(2) a direct nodewise subset bound;
(3) full R, D^P, D^N mixed masks;
(4) arbitrary and theorem-optimal scalar telescoping orders; and
(5) a source-resolved path sum.

The implementation takes a minimum only among independently valid upper bounds. It never takes the minimum of an empirical lower and a certificate.

7. Certified and empirical optimizer interfaces

The interval interface encloses explicit construction values with directed arithmetic. The exact symbolic small-case routine uses rational stationary point elimination and retains a nonnegativity witness. The optional SOS status reports import and solver availability but creates no certificate without a validated relaxation status.

The adversarial interface has two independent families: automatic differentiation with Adam followed by L-BFGS, and differential evolution. Optimization history records seed, restart, step, phase, attained ratio, and success. All values remain `EMPIRICAL_LOWER_BOUND` unless paired with an independent rigorous upper that closes the gap.

8. Experiment matrix and resource gate

Blocks A–I execute the base mathematical cells. Block J declares the extended seed/restart schedule. The resolved budget found 460,800 requested optimizer trajectories and an optimistic wall time of roughly sixteen hours, with a naive per-trajectory artifact layout producing hundreds of thousands of files. The resource resolution preserves every scientific axis, executes rigorous uppers and explicit lowers for each base cell, retains the full requested schedule in a resumable extension, and blocks release while that extension is pending.

This is a change in execution tier, not a silent reduction of the requested matrix. Required seeds and executed seeds have separate fields.

9. Artifact schema

Every run directory contains at least:

group	files
identity	`config.yaml`, `resolved_config.yaml`, `run_manifest.json`, `command.txt`
environment	`environment.json`, `hardware.json`, stdout/stderr logs
mathematical object	`tree.json`, `tree.svg`, `type_signature.json`, `law_tensors.npz`
evidence	local constants, reference metrics, optimization history, node contributions, final metrics, certificate
integrity	summary and `artifact_hashes.json`
extremizer extension	best lower, certified upper, gap, tensor, inputs, and independent recheck

The scientific-instance hash excludes seed and restart; the run identity does not. This prevents repeated executions of the same mathematical cell from being counted as independent experiments.

10. Figures and generated tables

The figure builder creates 18 main vector figures and eight supplementary topology atlases. Mathematical diagrams are compiled from TikZ; large data plots use a centralized colorblind-safe Matplotlib style. PDF is canonical and SVG is generated for documentation. Each figure-manifest entry records source artifacts, hashes, metric, unique instance count, seed/restart status, uncertainty, theorem reference, supported conclusion, and unsupported conclusion.

The table builder creates 16 mandatory LaTeX tables directly from Parquet/CSV indexes. No experimental value is typed into either manuscript.

11. Computational scaling

The current hardware is an NVIDIA RTX PRO 5000 Blackwell Generation Laptop GPU with approximately 24 GiB device memory. The registered invariant-control benchmark has exact reference/NumPy/CUDA parity and reports wall time, throughput, traced RAM, and allocated VRAM separately. Timing results are not used in mathematical claims.

12. Reproduction commands

The public entry points are:

```
.\scripts\run_tree_constants_v3_smoke.ps1
.\scripts\run_tree_constants_v3_exact.ps1
.\scripts\run_tree_constants_v3_full.ps1
.\scripts\run_tree_constants_v3_extended.ps1
.\scripts\resume_tree_constants_v3.ps1
.\scripts\build_tree_constants_v3_figures.ps1
.\scripts\build_tree_constants_v3_paper.ps1
.\scripts\audit_tree_constants_v3.ps1
.\scripts\release_tree_constants_v3.ps1
```

The canonical full command is intentionally fail-closed: it may regenerate all artifacts and PDFs successfully and still exit nonzero when novelty, human review, or the resource-gated extension remains unresolved.

13. PDF and release quality assurance

Both manuscripts are built by `latexmk` with nonstop interaction, halt-on-error, and file-line diagnostics. The audit scans undefined references/citations, duplicate labels, missing figures, and critical overfull boxes; Poppler renders every page to PNG, a contact sheet is created, and the final PDF is hashed. Four independent-role reviewer reports cover multilinear analysis, numerical certification, software reproducibility, and editorial/novelty risk.

14. Limitations

The suite is finite-dimensional, the full optimizer extension is pending, and general dense tensor operator norms may use Frobenius upper bounds. An immutable source commit can only be recorded after source generation and before the canonical rerun. Most importantly, reproducibility and software coverage cannot establish theorem-level novelty or replace accountable human approval.

15. Conclusion

V3 separates typed syntax, exact evaluation, recursive projection, upper certification, lower construction, empirical search, aggregation, publication output, and release governance. The base workflow is fully regenerable on the recorded environment. Its conservative status is a feature of the artifact contract: unresolved mathematical and publication conditions remain visible and block release.

References

1. Nicholas J. Higham, *Accuracy and stability of numerical algorithms*, 2 ed., Society for Industrial and Applied Mathematics, 2002.
2. Tamara G. Kolda and Brett W. Bader, *Tensor decompositions and applications*, SIAM Review **51** (2009), no. 3, 455–500.

Independent Researcher, Apizaco, Tlaxcala, Mexico

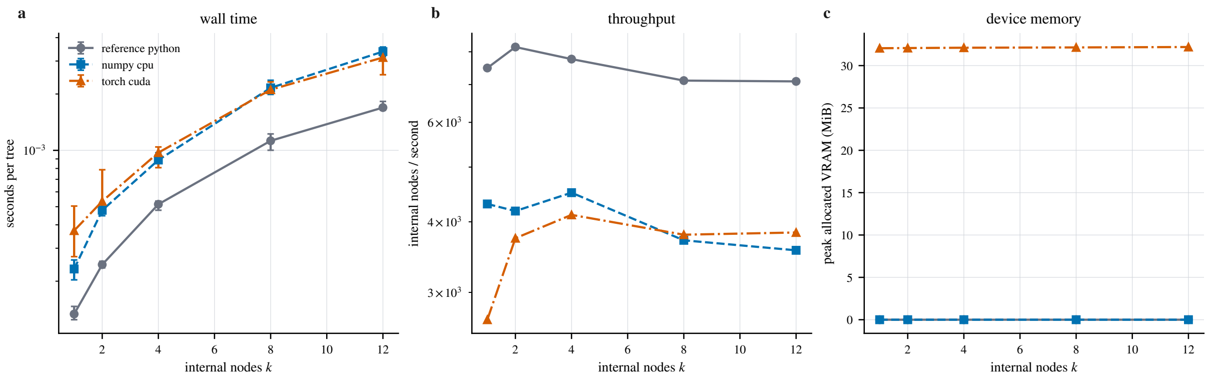

FIGURE 1. Registered reference/NumPy/CUDA scaling. The 15 backend/size cells use five timing repeats and 15–30 calls per repeat. Error bars are repeat min–max; the result is environment-specific and does not establish cross-hardware superiority.

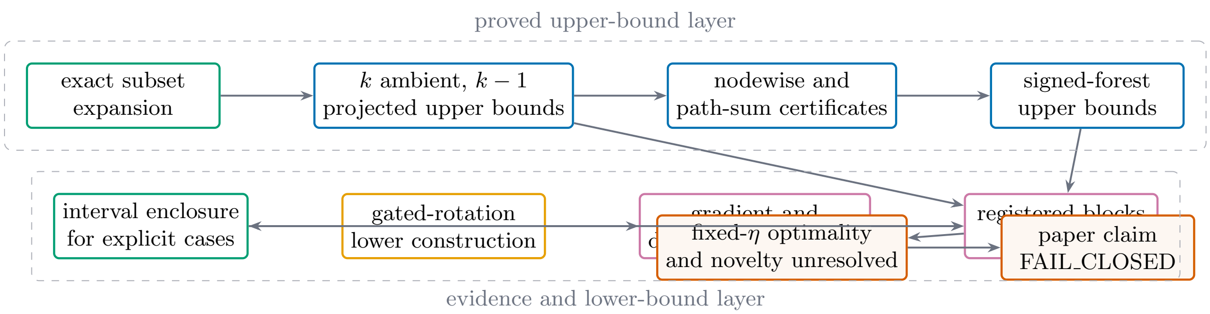

FIGURE 2. The claim/evidence DAG keeps proof, upper certificate, lower construction, empirical search, and release authorization in separate layers.

TABLE 3. Generated reproducibility manifest.

field	generated value
source commit	b718f4e51785
scientific instances A–I	15493
enumerated tree occurrences	81445
unique tree hashes	80870
leakage masks	1530
CPU/GPU max difference	1.922e-08
optimizer trajectories requested	460800
optimizer status	EXTENDED_PENDING_RESOURCE_GATE
release status	FAIL_CLOSED_NOVELTY